DENO

MANUAL INTRODUCTORIO DE DENO

Autor: Cristian Rodríguez Hidalgo

Curso: 2.º de Grado Superior en Desarrollo de Aplicaciones Web (DAW)

Fecha: 12 de octubre de 2026

Tecnología de ejecución para JavaScript, TypeScript y WebAssembly

2. ¿Qué es Deno y para qué sirve?

Deno es un entorno de ejecución. Dicho de forma sencilla, es un programa que permite ejecutar código JavaScript y TypeScript fuera del navegador, por ejemplo, desde la terminal del ordenador. También puede trabajar con WebAssembly. No es un lenguaje de programación nuevo: es una herramienta que permite ejecutar programas escritos con esos lenguajes.

Por ejemplo, JavaScript se usa normalmente para añadir interactividad a una página web. Con Deno también podemos ejecutar un archivo desde la terminal, crear un pequeño servidor web o preparar herramientas para un proyecto. Una ventaja para quien empieza es que Deno puede ejecutar archivos TypeScript directamente, sin tener que instalar por separado un compilador de TypeScript ni preparar un proceso de compilación para los casos básicos.

¿Qué problema intenta resolver?

Deno nació con la idea de hacer más sencillo y moderno el desarrollo con JavaScript fuera del navegador. Su creador, Ryan Dahl, ya había participado en la creación de Node.js. Deno propone una experiencia con herramientas incluidas y una configuración inicial reducida. Por ejemplo, incorpora comandos para ejecutar código, comprobar tipos, formatear archivos, revisar problemas de estilo y hacer pruebas.

Otra diferencia importante es la seguridad. De forma predeterminada, un programa de Deno no puede acceder libremente a archivos del ordenador, a la red o a variables de entorno. Si necesita hacerlo, hay que concederle permisos expresamente, por ejemplo, al ejecutarlo con una opción de permisos. Esto ayuda a evitar que un programa tenga acceso a más recursos de los necesarios, aunque no significa que cualquier programa sea automáticamente seguro.

¿Quién lo mantiene y cuál es su versión?

Deno es un proyecto de código abierto desarrollado por el equipo y la comunidad de Deno. La empresa Deno también participa en su desarrollo y en herramientas relacionadas. Ryan Dahl es uno de sus creadores y ha explicado públicamente los motivos originales del proyecto.

En la consulta realizada el 13 de octubre de 2026, la página oficial de versiones de GitHub mostraba Deno 2.9.2 como una versión publicada. Las versiones pueden cambiar, así que antes de instalarlo conviene consultar la página oficial de versiones para comprobar cuál es la estable más reciente.

Fuentes principales de este apartado: documentación oficial de Deno y publicación del equipo sobre el origen del proyecto.

3. Conceptos clave

Antes de empezar a utilizar Deno, es útil conocer estas ideas básicas. No hace falta dominarlas todas de memoria; lo importante es entender para qué sirve cada una.

3.1. Entorno de ejecución

Un entorno de ejecución es el programa que permite que nuestro código funcione. Por ejemplo, al escribir un archivo llamado hola.ts, Deno puede ejecutarlo desde la terminal. La terminal es la ventana donde escribimos comandos.

```
Ejemplo:  deno run hola.ts
```

Este comando indica a Deno que ejecute el archivo hola.ts. Si el archivo contiene una instrucción console.log("Hola"), se mostrará el texto en la terminal.

3.2. JavaScript y TypeScript

JavaScript es un lenguaje muy utilizado en páginas web y también puede ejecutarse en el servidor. TypeScript añade, entre otras cosas, la posibilidad de indicar qué tipo de dato esperamos en una variable o función. Esto puede ayudar a detectar errores mientras programamos. Deno permite ejecutar archivos TypeScript directamente.

```
function saludar(nombre: string): string { return "Hola, " + nombre; }
```

En este ejemplo, nombre debe ser un texto (string) y la función devuelve otro texto. Es una pequeña muestra de cómo TypeScript permite expresar los tipos de datos.

3.3. Permisos de seguridad

Deno limita por defecto algunas operaciones que pueden afectar al ordenador, como leer archivos o conectarse a la red. Si el programa necesita una de ellas, se debe conceder el permiso correspondiente. Por ejemplo, un servidor que escucha conexiones necesita permiso de red.

```
Ejemplo: deno run --allow-net servidor.ts
```

La opción --allow-net permite el acceso a la red. Es mejor conceder solo los permisos necesarios y no activar todos los permisos sin saber para qué sirven.

3.4. Herramientas incluidas

Deno incluye varias herramientas que en otros entornos pueden requerir programas adicionales. Entre ellas están el formateador, que ayuda a que el código tenga un formato ordenado; el linter, que señala posibles problemas de estilo o errores; y el ejecutor de pruebas, que permite comprobar que una parte del programa funciona como esperamos.

Por ejemplo, deno fmt sirve para dar formato a los archivos compatibles y deno test permite ejecutar pruebas automatizadas. Las pruebas son comprobaciones que se ejecutan para verificar el comportamiento del código.

3.5. Módulos y dependencias

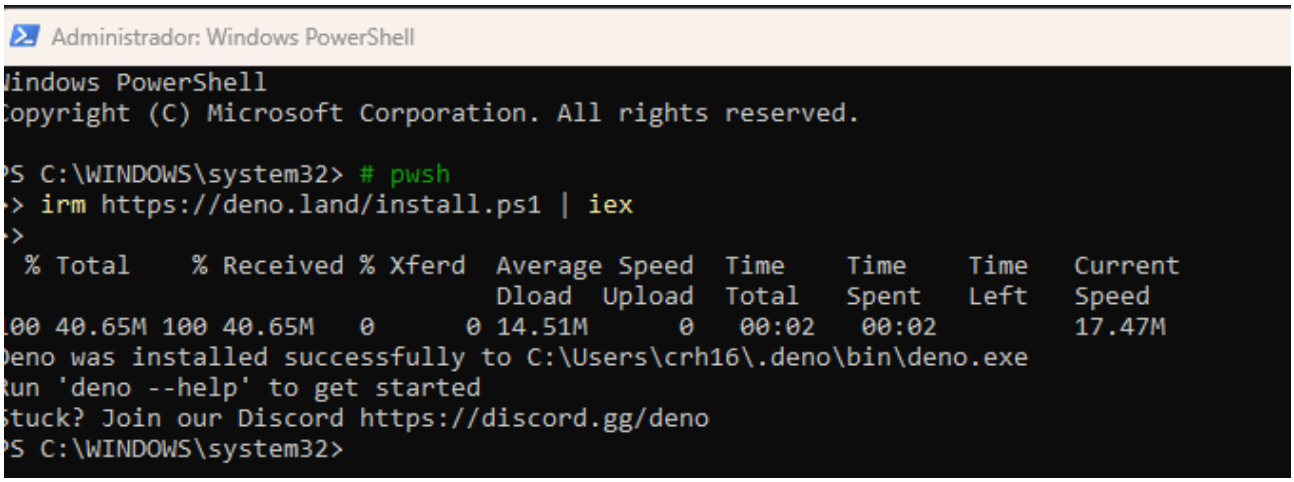
Un módulo es un archivo o conjunto de código que se puede reutilizar desde otro archivo. Para conectar módulos se usan instrucciones import y export. Las dependencias son paquetes de código creados por otras personas que nuestro proyecto utiliza. Deno permite usar paquetes de JSR y npm, aunque algunos paquetes pueden necesitar ajustes para funcionar correctamente.

```
import { algo } from "./utilidades.ts";
```

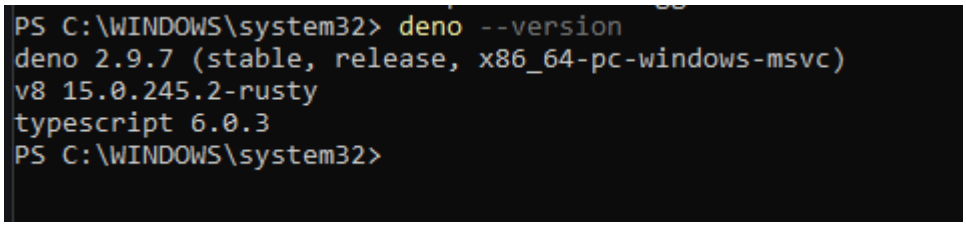
Esta línea es un ejemplo de importación desde otro archivo del proyecto. La ruta indica dónde se encuentra el módulo que queremos utilizar.

4.Requisitos e instalación

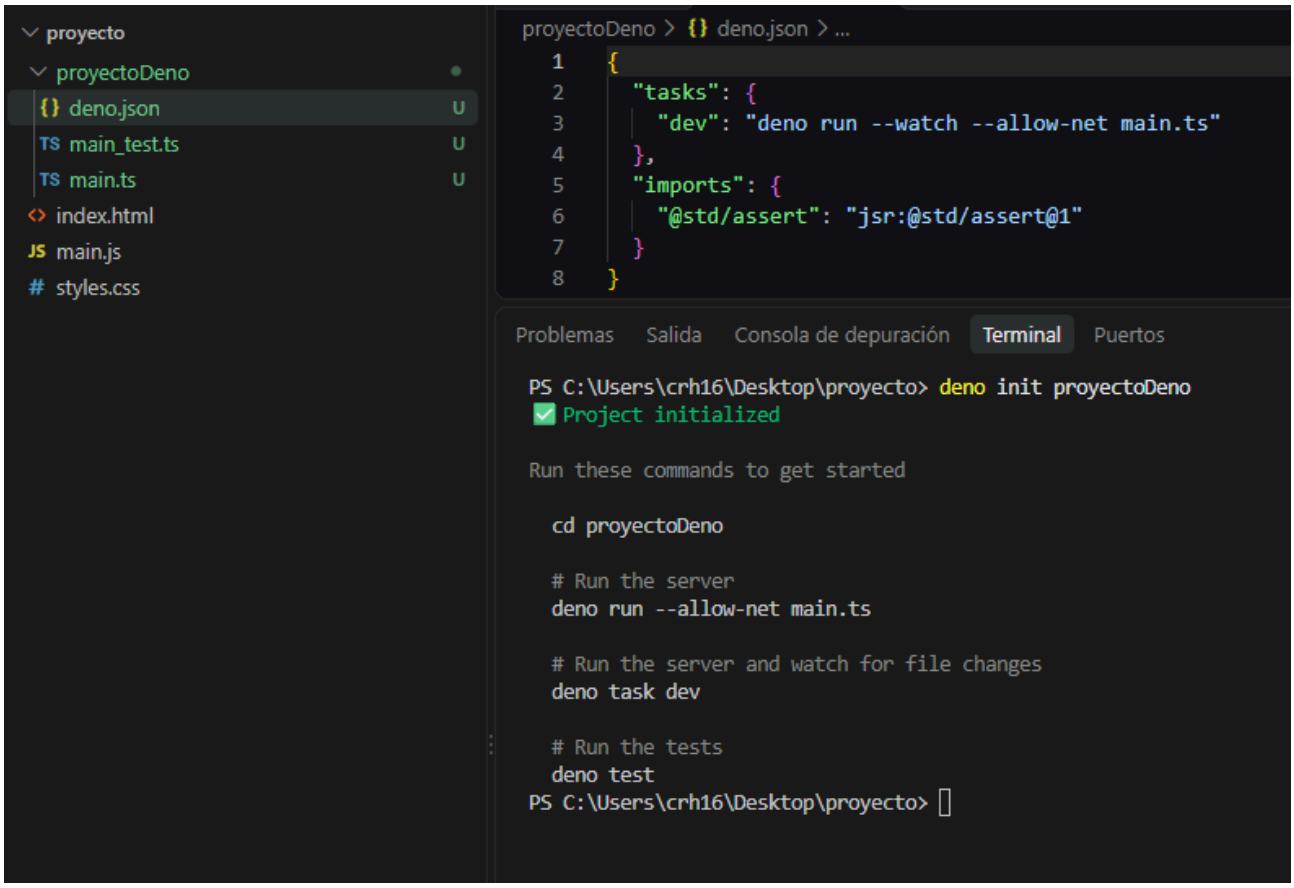
Comandos de instalación paso a paso, en bloques de código.



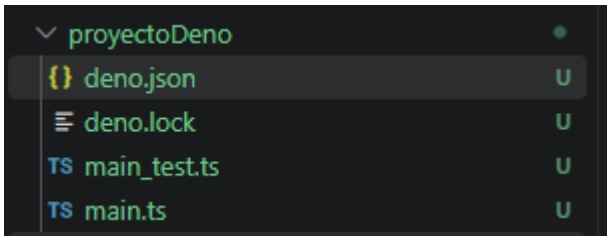
Verificamos la instalación



Una vez instalado con visual comprobamos que va poniendo lo siguiente y automáticamente se crea nuestro proyecto y la estructura de carpetas para un proyecto básico.



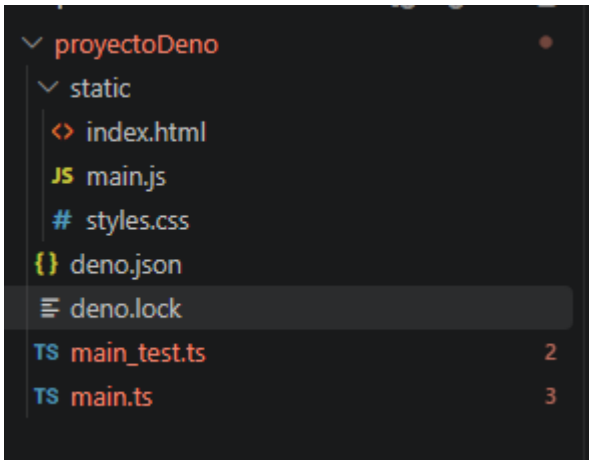
Estructura de carpetas



5. Ejemplo básico de uso (2–3 páginas)

1. **. Un mini proyecto propio: por ejemplo, una lista de tareas, un contador, un formulario o un catálogo sencillo.**

Estructura



En deno.json tenemos los tasks, y los imports.

Este es el equivalente al package.json de Node pero más ligero.

Los tasks son los comandos personalizados que ejecutamos con deno, el comando que estamos poniendo ahí nos permite ejecutar el programa y reiniciarlo cada vez que guardamos los cambios.

Los imports son las dependencias que vamos a necesitar para la ejecución de nuestro programa.

```
{
  "tasks": {
    "dev": "deno run --allow-net --allow-read --watch main.ts"
  },
  "imports": {
    "@std/assert": "jsr:@std/assert@1",
    "@std/http": "jsr:@std/http@1"
  }
}
```

Deno.lock.

Este es el archivo de bloqueo de dependencias en el que se guardan las versiones del proyecto para que cualquier persona que lo ejecute obtenga exactamente la misma versión, este se genera automáticamente.

main.ts

Es el punto de entrada de tu aplicación, escrito en TypeScript (Deno lo soporta de forma nativa, sin compilar ni configurar nada).

```ts
// Importamos serveDir: una función que entrega los archivos de una carpeta
// (index.html, main.js, styles.css) al navegador.
import { serveDir } from "@std/http/file-server";

// Lista donde guardamos las tareas. Cada tarea es un texto.
// Está en memoria: al reiniciar el servidor, se vacía.
const tareas: string[] = [];

// Esta función atiende cada petición que llega al servidor.
// Se exporta para poder usarla en main_test.ts.
export async function handler(req: Request): Promise<Response> {
  // Sacamos la ruta que pide el navegador (por ejemplo "/api/tareas")
  const url = new URL(req.url);

  // Si piden la lista (método GET), la devolvemos en formato JSON
  if (url.pathname === "/api/tareas" && req.method === "GET") {
    return Response.json(tareas);
  }

  // Si quieren añadir una tarea (método POST)
  if (url.pathname === "/api/tareas" && req.method === "POST") {
    // Leemos los datos que envía el navegador: { "texto": "..." }
    const datos = await req.json();

    // Guardamos el texto en la lista
    tareas.push(datos.texto);

    // Respondemos con el código 201, que significa "creado"
    return new Response("Tarea creada", { status: 201 });
```

```
  }

  // Para cualquier otra ruta, entregamos la página desde la carpeta "static"
  return serveDir(req, { fsRoot: "static" });
}

// Esto arranca el servidor solo cuando ejecutamos este archivo directamente
// (no cuando lo importa main_test.ts).
if (import.meta.main) {
  Deno.serve(handler);
}
```

Main_test.ts

Es el archivo de pruebas de main.ts. Deno tiene un *test runner* integrado, así que no necesitas instalar nada más.

```
// assertEquals compara dos valores: si son distintos, la prueba falla.
import { assertEquals } from "@std/assert";

// Importamos la función del servidor para probarla
import { handler } from "./main.ts";

// Deno.test crea una prueba: un nombre y el código que se ejecuta
Deno.test("POST crea una tarea y GET la devuelve", async () => {
  // Paso 1: simulamos que el navegador añade la tarea "Estudiar Deno"
  const crear = await handler(
    new Request("http://localhost/api/tareas", {
      method: "POST",
      body: JSON.stringify({ texto: "Estudiar Deno" }),
    }),
  );

  // Comprobamos que el servidor respondió 201 (creado)
  assertEquals(crear.status, 201);

  // Paso 2: pedimos la lista de tareas (GET)
  const respuesta = await handler(new Request("http://localhost/api/tareas"));
  const lista = await respuesta.json();

  // Comprobamos que la primera tarea de la lista es la que añadimos
  assertEquals(lista[0], "Estudiar Deno");
});
```

main.js
Es el código que se ejecuta en el navegador y hace que la página funcione.

```
// Guardamos la lista (<ul>) de la página para poder añadirle tareas
const lista = document.getElementById("lista");

// Pide las tareas al servidor y las muestra en la página
async function cargar() {
  // Pedimos la lista al servidor y esperamos la respuesta
  const respuesta = await fetch("/api/tareas");
  const tareas = await respuesta.json();

  // Vaciamos la lista para no repetir tareas
  lista.innerHTML = "";

  // Por cada tarea, creamos un <li> y lo metemos en la lista
  for (const texto of tareas) {
    const li = document.createElement("li");
    li.textContent = texto;
    lista.append(li);
  }
}

// Se ejecuta cuando pulsamos el botón "Añadir"
document.getElementById("formulario").onsubmit = async (e) => {
  // Evita que la página se recargue al enviar el formulario
  e.preventDefault();

  // Leemos lo que hemos escrito en el input
  const texto = document.getElementById("texto").value;

  // Enviamos el texto al servidor
  await fetch("/api/tareas", {
    method: "POST",
    body: JSON.stringify({ texto: texto }),
  });

  // Vaciamos el input
  e.target.reset();

  // Volvemos a pedir la lista para ver la tarea nueva
```

```
  cargar();
};

// Al abrir la página, mostramos las tareas que ya existan
cargar();
```

Resultado final



La prueba con el test desde el terminal:

```
PS C:\Users\crh16\Desktop\optativa\proyectoDeno> deno test
running 1 test from ./main_test.ts
POST crea una tarea y GET la devuelve ... ok (2ms)

ok | 1 passed | 0 failed (6ms)

PS C:\Users\crh16\Desktop\optativa\proyectoDeno>
```

ERROR FORZADO

```
PS C:\Users\2DAW\Desktop\Optativa\proyectoDeno> deno test
Check main_test.ts
running 1 test from ./main_test.ts
POST crea una tarea y GET la devuelve ... FAILED (4ms)

ERRORS

POST crea una tarea y GET la devuelve => ./main_test.ts:8:6
error: AssertionError: Values are not equal.


    [Diff] Actual / Expected


-   Estudiar Deno
+   Estudiar


  throw new AssertionError(message);
        ^
    at assertEquals (https://jsr.io/@std/assert/1.0.19/equals.ts:67:9)
    at file:///C:/Users/2DAW/Desktop/Optativa/proyectoDeno/main_test.ts:25:3

FAILURES

POST crea una tarea y GET la devuelve => ./main_test.ts:8:6

FAILED | 0 passed | 1 failed (13ms)

error: Test failed
```

Solución

```
PS C:\Users\2DAW\Desktop\Optativa\proyectoDeno> deno test
running 1 test from ./main_test.ts
POST crea una tarea y GET la devuelve ... ok (3ms)

ok | 1 passed | 0 failed (8ms)
```

6. Ventajas, inconvenientes y conclusión

Ventajas

- Es relativamente sencillo empezar: puede ejecutar archivos JavaScript y TypeScript con comandos directos.
- TypeScript está integrado, por lo que no hace falta instalar un compilador aparte para empezar con ejemplos sencillos.

- Incluye herramientas útiles como el formateador y el ejecutor de pruebas.
- Su sistema de permisos limita el acceso a archivos, red y variables de entorno hasta que se autoriza.
- Puede utilizar muchos paquetes de npm y trabajar con proyectos que ya usan package.json.

Inconvenientes

- Si ya estamos acostumbrados a Node.js, algunos comandos y formas de configurar el proyecto pueden resultar diferentes.
- Aunque admite muchos paquetes de npm, no todos funcionan siempre sin cambios; algunos dependen de características específicas de Node.js.
- El sistema de permisos requiere entender qué acceso necesita cada programa. Si falta un permiso, el código puede detenerse con un error.
- Hay muchos tutoriales antiguos en Internet, por lo que es importante comprobar que los pasos coinciden con la versión que estamos usando.

Conclusión y valoración personal

Como estoy empezando en Desarrollo de Aplicaciones Web, considero que Deno puede ser una herramienta interesante para aprender a ejecutar JavaScript y TypeScript fuera del navegador. Me parece útil que incluya varias herramientas desde el principio y que los permisos de seguridad estén limitados por defecto. Eso sí, tendría que practicar los comandos y revisar la compatibilidad antes de utilizarlo en un proyecto grande.

7. Recursos y bibliografía

Las siguientes páginas web se han utilizado para preparar este manual. Se ha dado prioridad a la documentación oficial de Deno. Los enlaces se incluyen para poder consultar la información completa y comprobar posibles cambios.

Documentación oficial: página de inicio y primeros pasos

https://docs.deno.com/runtime/

Definición general de Deno, características principales y guía para empezar.

Documentación oficial: seguridad y permisos

https://docs.deno.com/runtime/fundamentals/security/

Explica por qué Deno limita por defecto el acceso a archivos, red y variables de entorno.

Documentación oficial: soporte de TypeScript

https://docs.deno.com/runtime/fundamentals/typescript/

Describe cómo ejecutar TypeScript y cómo comprobar los tipos.

Documentación oficial: ejecución de código

https://docs.deno.com/runtime/run/

Comandos para ejecutar archivos y conceder permisos.

Documentación oficial: módulos

https://docs.deno.com/runtime/fundamentals/modules/

Explica importaciones, exportaciones y los módulos de JavaScript.

Documentación oficial: compatibilidad con Node.js y npm

https://docs.deno.com/runtime/fundamentals/node/

Información sobre compatibilidad con paquetes npm y proyectos de Node.js.

Blog oficial de Deno: Announcing the Deno Company

https://deno.com/blog/the-deno-company

Artículo de Ryan Dahl y Bert Belder sobre el proyecto y sus objetivos.

Versiones oficiales de Deno en GitHub

https://github.com/denoland/deno/releases

Página para comprobar las versiones publicadas y consultar sus cambios.

Fecha de consulta de los recursos: 13 de octubre de 2026.

8. Enlace GitHub

https://github.com/crodhid/manual-deno-rodr-guez